

Feeding & Crafting Mechanic

Valentino Clerx · Fontys Hogeschool, Master of Applied IT · 2026

1. Purpose

This document analyses the design decisions behind the feeding and crafting mechanic used in the Game-Based Learning study on leopard gecko husbandry. It covers ingredient taxonomy, recipe progression rationale, system architecture, and validation logic. A summary of this analysis is referenced in the accompanying devlog.

2. Learning Objectives

The Feeding module must teach three discrete concepts drawn directly from the ReptiFiles Leopard Gecko Care Sheet, which underpins all four experimental conditions:

#	Concept	Quiz Item Type
1	Leopard geckos are strict insectivores — fruit and vegetables are harmful.	Factual recall
2	Dietary variety across feeder insect species is essential for nutritional balance.	Applied judgement
3	Calcium and multivitamin supplementation prevents metabolic bone disease.	Factual recall

3. Ingredient Taxonomy

Five ingredients populate the feeding scene. Three are correct components of a valid gecko meal; two are deliberate distractors that probe objective 1 (the insectivore rule). Each ingredient is assigned a distinct colour so the bowl's blended fill provides passive visual feedback without requiring the player to read tooltips.

Ingredient	Colour (hex)	Role	Concept Targeted
Calcium Powder	#FFFFFF (White)	Valid — supplement	Objective 3
Cricket	#996633 (Brown)	Valid — primary feeder	Objectives 1 & 2
Mealworm	#FFCC66 (Golden)	Valid — secondary feeder	Objective 2
Banana	#FFF266 (Pale Yellow)	Distractor — fruit	Objective 1
Cucumber	#80E64D (Green)	Distractor — vegetable	Objective 1

Banana and Cucumber were selected specifically because they are common household foods that novice owners may plausibly consider offering, making the distractor choice ecologically valid. A player who selects either ingredient and triggers the fail path will, in the LLM-interactive condition, receive a contextual explanation from the AI veterinarian about why fruit and vegetables are harmful, reinforcing objective 1 through negative feedback rather than mere trial and error.

4. Recipe Progression and Scaffolding

Three recipes are defined across the game's loop structure. The progression is intentional: each loop revisits the feeding task at a higher conceptual level, implementing the intentional repetition principle identified in the GBL literature (Conmy, 2023; Talan et al., 2020). Presenting all three concepts

simultaneously in the first encounter would impose excessive cognitive load on novice players, particularly given the 20 to 30 minute study window.

Loop	Recipe	Ingredients	New Concept Introduced
Loop 1	Loop 1 Dish	Cricket · Cricket · Cricket	Insects are the only valid food (Obj. 1)
Loop 2	Loop 2 Dish	Cricket · Mealworm · Mealworm	Variety across feeder species (Obj. 2)
Loop 3	Leopard Gecko Dish	Calcium Powder · Cricket · Mealworm	Supplementation required (Obj. 3)

The final recipe — used in the static condition and as the culminating task in the looped conditions — requires all three correct ingredient types and is the recipe assessed by the post-test quiz items. The active recipe is set at runtime by `GameFlowManager.loop_dish`, allowing the same Bowl scene to serve all loop stages without code duplication.

5. System Architecture

The diagram below illustrates the full data and signal flow for the feeding mechanic, from ingredient drag-and-drop through recipe validation to the success and fail outcomes.

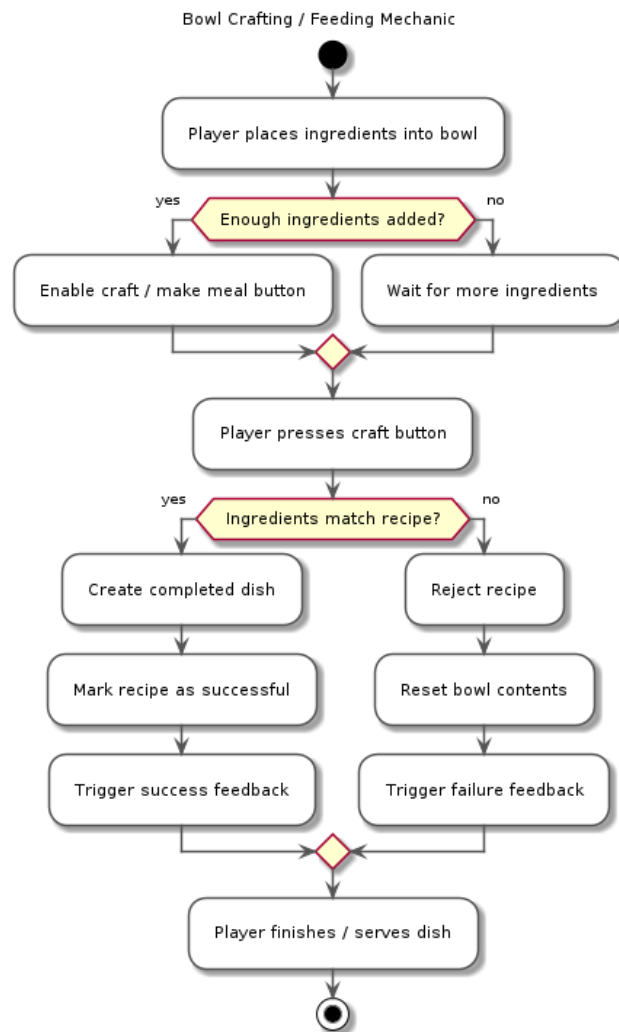


Figure 1. Feeding mechanic system overview. Dashed lines indicate Godot signals. All nodes are part of the same scene tree; `GameFlowManager` is an autoload singleton.

6. Validation Logic

Recipe checking is performed by sorting both the current ingredient name list and the target recipe array alphabetically before comparing them. This makes the check order-independent, so players are not penalised for dragging ingredients in a different sequence than the recipe definition. The sort-and-compare pattern is preferable to a positional check because it keeps the interaction model forgiving without reducing the educational challenge of selecting the correct three items.

```
func _on_cook_button_pressed(): var current_names = [] for node in ingredient_nodes:
current_names.append(node.ingredient_name) current_names.sort() var required =
RECIPES[active_recipe].duplicate() required.sort() if current_names == required:
AudioManager.play_correct_dish_sfx() finish_dish() else:
AudioManager.play_wrong_dish_sfx() fail_dish()
```

7. Success and Fail Paths

On a successful match, all ingredient nodes are locked (`input_pickable = false`) and `vet_manager.on_dish_succes` is called. In the LLM-interactive condition this gives the AI veterinarian the opportunity to deliver a contextual explanation of why the chosen combination is correct, connecting the in-game action to real-world husbandry reasoning. In the hardcoded condition the same signal triggers a fixed authored response from the dialogue tree. The `is_static` export flag controls whether `recipe_completed` is emitted immediately or deferred to an explicit finish button press, supporting both looped practice rounds and single-submission scenarios within the same Bowl scene.

On failure, every ingredient node is returned to its spawn position via `node.reset_fail()` and the ingredient list is cleared. The Make Meal button is hidden and ingredients are re-enabled for dragging. `vet_manager.on_dish_fail` is called in parallel, allowing the AI to offer corrective guidance. The bowl state can also be reconstructed from a saved name list via `restore_ingredient_names()`, which will support session-level state persistence for the between-condition study design.

8. Bowl Colour Feedback

`show_filled_visual()` averages the `INGREDIENT_COLORS` entries for all nodes currently in the bowl and applies the result to the `filled_sprite` modulate property. This gives the player a continuously updating visual signal about their current selection without exposing the recipe answer. A mix of brown, golden, and white (cricket, mealworm, calcium powder) produces a pale tan colour, visually distinct from any combination that includes the fruit or vegetable distractors. The effect is intentionally subtle: it rewards attentive players without substituting for deliberate ingredient reasoning.

9. Cross-Condition Considerations

The Bowl scene is structurally identical across all three game conditions. What differs is the knowledge resource available to the player when deciding which ingredients to add. In the Game-Non-Interactive condition an in-game caresheet is available but no NPC guidance is offered. In the Game-Hardcoded condition the vet NPC answers a fixed set of pre-authored questions via an RPG-style dialogue tree. In the Game-LLM-Interactive condition the same NPC is backed by a Google Gemini instance constrained to the ReptiFiles caresheet domain, allowing open-ended natural language queries. The bowl mechanic and validation logic are shared; only the `vet_manager` target varies per condition, keeping the experimental manipulation clean.